

```
+++  
date = '2026-04-15T06:00:00+08:00'  
categories = ['interview', 'cpp']  
tags = ['八股', '面试', 'cpp']  
title = 'Basic'  
+++
```

C 和 C++ 的区别（总结版）

2.1.1 设计目标的不同

C —— “高效运行”

- 目标：贴近硬件、提供最小抽象、保证可移植性
- 主要用于系统级开发（OS、驱动、嵌入式）

C++ —— “高效 + 可维护”

- 目标：在不损失性能的前提下提供更强的抽象能力
- 支持面向对象、泛型、RAII、模板元编程等

一句话：

C 关注“能不能跑”，C++ 关注“能不能长期维护”。

2.1.2 编程范式差异（核心区别）

C：面向过程（Procedure-Oriented）

- 数据与函数分离
- 通过函数操作结构体

```
struct Point { int x, y; };  
void move(struct Point* p);
```

C++：面向对象（Object-Oriented）

- 数据 + 行为封装在类中
- 支持继承、多态、封装

```
class Point {  
public:  
    void move();  
};
```

C++ 同时支持：

- 面向对象
- 泛型编程（模板）
- 函数式编程（lambda）
- 元编程（模板元编程）

2.1.3 抽象能力对比

C

- 结构体 + 函数
- 手动管理复杂度

C++

- 类、继承、多态
- 模板（泛型）
- STL（容器 + 算法）
- RAII 自动管理资源

抽象能力远强于 C。

2.1.4 内存与资源管理

C：手动管理内存

```
malloc();  
free();
```

容易出现：

- 内存泄漏
- 野指针
- 重复释放

C++: RAII + 智能指针

```
auto p = std::make_unique<int>(10);
```

- 自动释放
- 异常安全
- 所有权语义明确

2.1.5 类型安全

C

- 类型检查弱
- 隐式转换多
- 容易写出未定义行为

C++

- 强类型系统
- 重载、模板、const、引用
- 编译期检查更严格

2.1.6 标准库能力

C 标准库 (libc)

- 字符串处理
- 数学函数
- IO
- 较小

C++ 标准库 (STL + 现代库)

- 容器 (vector、map、list...)
- 算法 (sort、find...)
- 智能指针
- 正则表达式
- 多线程库
- chrono 时间库

C++ 标准库远比 C 强大。

2.1.7 代码规模与维护性

C

- 适合小型、底层系统
- 大型项目维护成本高

C++

- 适合中大型工程
- 抽象能力强，易扩展、易维护

2.1.8 性能对比

- **C++ $\approx$ C**
- C++ 的抽象在编译期展开（模板、inline）
- 遵循 **零开销抽象（Zero-overhead abstraction）**

写得好的 C++ 性能不输 C；写得差的 C++ 也能很慢。

2.1.9 为什么有了 C 还要 C++（总结）

- C 解决“能不能高效跑”
- C++ 解决“能不能长期维护和扩展”
- C 不擅长组织复杂系统
- C++ 在不牺牲性能的前提下提供：
 - 更强抽象能力
 - 更好的可维护性
 - 更安全的资源管理

一句话：

C 是工具箱，C++ 是工程体系。

2.1.10 实际使用建议（工程视角）

- 底层驱动、裸机开发、MCU $\rightarrow$ C
- 中大型系统、复杂逻辑、需要抽象 $\rightarrow$ C++
- 性能敏感但需要安全性 $\rightarrow$ 现代 C++（RAII + 智能指针）
- 团队协作、大型项目 $\rightarrow$ C++ 更合适

下面我把你这段“C++ 的三大特性 / 四大特性 + 继承 + C/C++ 编译差异”全部整理成一份 **结构清晰、适合八股文/面试的高质量总结版**。

内容会更系统、更紧凑、更容易背诵，且保持你要求的 Markdown 最大二级标题。

C++ 的三大特性（或四大特性）

C++ 是面向对象语言，其核心特性通常总结为 **三大特性：封装、继承、多态**。
有时也会加上 **抽象**，称为四大特性。

2.2.1 三大特性（经典三大）

封装（Encapsulation）

定义

将 **数据 + 操作数据的函数** 封装在类中，并通过访问权限隐藏内部实现。

作用

- 隐藏实现细节
- 控制访问权限
- 提高安全性与可维护性

实现方式

- `class`
- `private` / `protected` / `public`

```
class MyClass {  
private:  
    int data;  
public:  
    void setData(int v) { data = v; }  
    int getData() { return data; }  
};
```

继承 (Inheritance)

定义

派生类复用基类的属性和方法，并可扩展或重写。

作用

- 代码复用
- 扩展性强
- 多态的基础

实现方式

```
class Base { };  
class Derived : public Base { };
```

多态 (Polymorphism)

定义

同一接口，不同对象表现不同行为。

分类

- **编译时多态**：函数重载、运算符重载
- **运行时多态**：虚函数 (virtual)

实现方式

```
class Base {  
public:  
    virtual void show() { cout << "Base"; }  
};  
class Derived : public Base {  
public:  
    void show() override { cout << "Derived"; }  
};
```

2.2.2 四大特性（增加“抽象”）

抽象（Abstraction）

定义

隐藏复杂实现，仅暴露必要接口。

作用

- 降低复杂度
- 提高可扩展性
- 让用户关注“做什么”，而不是“怎么做”

实现方式：抽象类 + 纯虚函数

```
class Shape {  
public:  
    virtual void draw() = 0; // 纯虚函数  
};
```

2.2.3 总结（面试高频）

- **封装**：隐藏细节，保护数据
- **继承**：代码复用，扩展功能
- **多态**：同一接口多种行为
- **抽象**：隐藏复杂性，暴露接口

一句话：

封装是基础，继承是手段，多态是目的，抽象是思想。

2.3 继承是什么？

2.3.1 定义

继承（Inheritance）是 OOP 的核心机制，表示 **派生类 is-a 基类**。

作用：

- 代码复用
- 扩展功能
- 实现多态

2.3.2 基本语法

```
class Base { };  
class Derived : public Base { };
```

2.3.3 继承方式（访问控制）

| 继承方式 | 基类 public 成员 | 基类 protected 成员 | 基类 private 成员 |
|--|--|
| public | public | protected | 不可访问 |
| protected | protected | protected | 不可访问 |
| private | private | private | 不可访问 |

2.3.4 继承的作用（重点）

1. 代码复用
2. 可扩展性
3. 支持多态

2.3.5 继承与多态的关系

- 继承是多态的基础
- 虚函数 + 继承 → 运行时多态

```
Base* p = new Derived();  
p->func(); // 调用 Derived::func()
```

2.3.6 面试总结句

继承让子类拥有父类特性，并可扩展或重写，是实现多态的基础。

2.4 C++ 编译时与 C 的不同，在 C++ 中如何使用 C？

2.4.1 C++ 编译时与 C 的不同

项目	C	C++
	--	
名字修饰	无	有 (Name Mangling)
类型检查	弱	强
函数重载	不支持	支持
模板	无	有
内联/constexpr	基础	更强

2.4.2 C++ 编译时机制

- 名字修饰 (Name Mangling)：区分重载函数
- 模板实例化：编译期生成代码
- 更严格的类型检查
- 更多编译期优化 (inline、constexpr)

2.4.3 在 C++ 中使用 C 代码

方法 1：直接包含 C 头文件

```
#include <stdio>
#include <string>
```

方法 2：使用 extern "C" 防止名字修饰

```
extern "C" {
    #include "my_c_header.h"
}
```

或：

```
extern "C" void foo(int);
```

作用：

告诉 C++ 按 C 的方式生成符号名，避免链接错误。

2.4.4 注意事项

1. C 头文件要用 `extern "C"` 包裹
2. C 不能使用 C++ 特性（类、模板、重载）
3. 注意类型兼容性（如 `bool`、引用等 C 不支持）

2.4.5 面试总结句

C++ 编译期比 C 更强，支持名字修饰和模板；C 代码通过 `extern "C"` 可在 C++ 中无缝调用。

extern 关键字

基本作用

`extern` 的核心作用：

- 声明一个在其他文件中定义的变量或函数
- 不分配存储空间
- 建立外部链接（External Linkage）
- 用于 跨文件访问全局变量或函数

一句话：

`extern` = 声明，不定义；告诉编译器“这个符号在别处定义”。

extern 修饰全局变量（最常见）

文件 A（定义）：

```
// a.c
int g_val = 10;    // 定义 + 分配空间
```

文件 B（声明）：

```
// b.c
extern int g_val; // 声明，不分配空间
```

特点：

- 编译阶段只检查声明是否存在
- 链接阶段找到真正的定义
- 多文件工程标准写法

extern 与头文件（规范用法）

头文件（只放声明）

```
// config.h
extern int g_val;
```

源文件（放定义）

```
// config.c
int g_val = 10;
```

规范：

- 头文件只放 **extern** 声明
- 定义只能出现一次

避免多重定义错误。

extern 与函数

```
extern void foo(int x);
```

但：

对函数来说 **extern** 可以省略，因为函数默认具有外部链接。

也就是说：

```
void foo(int x); // 等价于 extern void foo(int x);
```

extern 与 static（高频考点）

```
static int a = 10;    // 内部链接（只能本文件使用）
extern int a;         // 外部链接（跨文件）
```

结论：

static 与 extern 互斥。

static 让符号只在本文件可见，extern 让符号跨文件可见。

extern 与 const（易错点）

```
// file1.c
const int g_val = 10;    // const 全局变量默认内部链接

// file2.h
extern const int g_val; // 必须加 extern 才能跨文件访问
```

关键点：

- C 中 const 全局变量默认是内部链接（相当于 static）
- 想跨文件访问必须加 extern

常见错误示例（面试必问）

错误写法：头文件直接定义变量

```
// config.h
int g_val;    // 错误！会导致 multiple definition
```

多个 .c 文件包含该头文件 → 链接时报错：

```
multiple definition of `g_val'
```

正确写法：

```
// config.h
extern int g_val;

// config.c
int g_val = 10;
```

嵌入式开发中的典型用途

- 全局配置参数
- 外设句柄（如 UART_HandleTypeDef）
- RTOS 任务句柄
- 跨模块共享状态

例如：

```
extern UART_HandleTypeDef huart1;
```

注意点（总结）

- extern 是 **声明**，不是定义
- 不分配内存
- 用于跨文件访问变量或函数
- 头文件只放 extern 声明
- const 全局变量跨文件必须加 extern
- static 与 extern 互斥（内部链接 vs 外部链接）

总结

extern 用于跨文件声明变量或函数，只做声明不分配内存；头文件放 extern，源文件放定义；
const 全局变量默认内部链接，跨文件必须加 extern。

public 、 protected 、 private 的访问控制

public 成员可以在任何地方被访问，包括类外部的任何代码。

protected 成员只能在类内部和派生类中访问，但不能在类外部直接访问。

private 成员只能在类的内部访问，不能在外部代码中直接访问。派生类也不能直接访问基类的 private 成员，除非通过公共接口访问。

使用场景

封装性和数据安全

通常，我们将类的成员声明为 `private`，并通过公有（`public`）的 `getter` 和 `setter` 方法来控制对成员的访问，从而确保数据不被随意修改。

继承和扩展性

当我们设计继承体系时，可以将一些方法或数据成员声明为 `protected`，使得派生类能够继承和修改这些成员，而外部无法直接访问。

下面我将你给出的“**常量成员函数 + new 和 malloc 区别**”重新整理成 **最大标题为二级、结构清晰、逻辑紧凑、适合八股文/面试的高质量总结版**。

内容经过优化，更容易背诵，也更符合工程面试表达。

常量成员函数（const member function）

定义

常量成员函数是在成员函数声明末尾加上 `const`，表示该函数 **不会修改对象状态**。

```
class A {  
    int x;  
public:  
    int getX() const { return x; }  
};
```

核心含义：

- 保证不修改成员变量
- 可以被 `const` 对象调用

语法与特点

返回类型 函数名(参数) `const`;

特点：

1. this 指针类型改变

- 普通成员函数：A* const this
- const 成员函数：const A* const this
- → 不能修改成员变量，也不能调用非 const 成员函数

2. 可被 const 对象调用

```
const A a;  
a.getX(); // OK
```

示例

```
class A {  
    int x;  
public:  
    int get() const { return x; } // const 成员函数  
    void set(int v) { x = v; }   // 非 const  
};
```

用途

- 保证函数不修改对象，提高安全性
- 支持 const 对象、const 引用调用
- 明确区分“读操作”和“写操作”

注意事项

1. const 成员函数 **不能修改普通成员变量**
2. 但可以修改 mutable 成员
3. const 成员函数 **只能调用其他 const 成员函数**

```
class A {  
    mutable int cnt;  
public:  
    int get() const {  
        cnt++; // OK, mutable  
        return cnt;  
    }  
};
```

面试总结句

`const` 成员函数保证不修改对象状态，可被 `const` 对象调用，是区分读写操作的重要机制。

new 和 malloc 的区别

语言层级不同

- `malloc` : C 语言库函数
- `new` : C++ 运算符（语言级特性）

返回类型不同（重要）

- `malloc` 返回 `void*`，需要强制转换
- `new` 返回具体类型指针，无需转换

```
int* p = (int*)malloc(sizeof(int));
```

```
int* p = new int;
```

是否调用构造/析构函数（核心区别）

- `malloc` : 不调用构造函数
- `free` : 不调用析构函数
- `new` : 调用构造函数
- `delete` : 调用析构函数

这是 C++ 中最关键的区别。

内存初始化行为

- `malloc` : 不初始化内存
- `new` :
 - `new int` : 不初始化
 - `new int()` : 初始化为 0
 - `new Obj()` : 调用构造函数

释放方式不同（必须配对）

- malloc → free
- new → delete
- new[] → delete[]

错误配对会导致未定义行为。

失败处理方式

- malloc：失败返回 NULL
- new：失败抛出 std::bad_alloc 异常

```
try {  
    int* p = new int[100000000000];  
} catch (std::bad_alloc& e) {  
    // handle  
}
```

分配机制不同

- malloc
 - 只分配内存
 - 不关心类型
 - 不执行构造/析构
- new
 - 分配内存 + 调用构造函数
 - 类型安全
 - C++ 语义更完整

使用场景建议

- C 代码或需要手动管理内存 → malloc/free
- C++ 对象、构造/析构、异常安全 → new/delete
- 现代 C++：优先使用智能指针（make_unique/make_shared）

注意点

- malloc/free 与 new/delete **不能混用**
- 数组必须使用 new[] / delete[]

- C++ 中应优先使用 `new` 或智能指针，而不是 `malloc`

面试总结句

`malloc` 只分配内存，`new` 负责分配内存并调用构造函数；`free` 不调用析构，`delete` 会调用析构；`new` 类型安全，`malloc` 需要强转。

C++ 中 `enum class` 与传统 `enum` 的区别

作用域 (Scope)

传统 `enum`

- 枚举成员直接暴露在所在作用域
- 容易产生命名冲突

```
enum Color { Red, Green, Blue };  
int x = Red; // 直接使用
```

`enum class`

- 枚举成员属于枚举类型的作用域
- 必须通过类型名访问，避免冲突

```
enum class Color { Red, Green, Blue };  
int x = Color::Red; // 必须加作用域
```

类型安全 (Type Safety)

传统 `enum`

- 可隐式转换为 `int`
- 容易出现错误比较

```
enum Color { Red };
int x = Red;    // OK
if (Red == 1)  // 也 OK (危险)
```

enum class

- 强类型枚举，不允许隐式转换
- 必须显式转换

```
enum class Color { Red };
int x = Color::Red;  // ❌ 错误
int y = static_cast<int>(Color::Red); // ✔
```

底层类型（Underlying Type）

传统 enum

- 默认底层类型为 int
- C++11 后可指定：

```
enum Color : uint8_t { Red, Green };
```

enum class

- 默认也是 int
- 更常用于显式指定底层类型（节省空间、用于状态机）

```
enum class State : uint8_t { Idle, Run, Stop };
```

与整数的交互

特性	enum	enum class
隐式转 int	✔	❌

特性	enum	enum class
与整数比较	✓	✗
需要显式转换	✗	✓

使用场景

传统 enum

- 数值含义明确
- 不需要强类型保护
- C 风格接口、寄存器值、协议字段

enum class（推荐）

- 状态机（State Machine）
- 事件类型（Event Type）
- 避免命名冲突
- 需要强类型检查的大型工程

总结对比（面试可背诵）

特性	enum	enum class
作用域	无	有
类型安全	弱	强
隐式转 int	✓	✗
底层类型	默认 int，可指定	默认 int，可指定
推荐场景	简单枚举、C 接口	状态机、事件、强类型场景

一句话：

enum class = 强作用域 + 强类型 + 可控底层类型，是现代 C++ 推荐的枚举方式。

C++ 编译的四个阶段

预处理 (Preprocessing)

- 展开 `#include`
- 替换 `#define`
- 处理条件编译
- 删除注释

输出: `.i`

编译 (Compilation)

- 语法分析、语义分析
- 优化
- 生成汇编代码

输出: `.s`

汇编 (Assembly)

- 汇编器将 `.s` 转为机器码

输出: 目标文件 `.o`

链接 (Linking)

- 符号解析
- 重定位
- 合并目标文件与库

输出: 可执行文件 `.elf / .exe`

一句话:

预处理 → 编译 → 汇编 → 链接。

左值引用与右值引用

左值引用（T&）

- 绑定可寻址对象（变量）
- 常用于函数参数、返回值优化

```
int a = 10;
int& r = a;
```

右值引用（T&&）

- 绑定临时对象、将亡值
- 支持移动语义、完美转发

```
std::string&& s = std::string("abc");
```

区别总结

特性	左值引用	右值引用
绑定对象	左值	右值
是否可修改	✓	✓
用途	别名、传参	移动语义、转移资源

一句话：

左值引用是别名，右值引用是“可转移资源的临时对象”。

C++ 的几种传参方式

值传递（Pass by Value）

- 拷贝实参
- 修改不影响原值

- 适合小对象

指针传递（Pass by Pointer）

- 传地址副本
- 可修改原对象
- 可能为 nullptr

引用传递（Pass by Reference）

- 无拷贝
- 不会为空
- 语义自然（推荐）

const 引用传递（最常用）

- 不拷贝
- 不可修改
- 可绑定临时对象
- 大对象最佳选择

右值引用传递

- 用于移动语义
- 常见于构造函数、容器

传参方式总结（面试可背诵）

方式	是否拷贝	是否可修改	是否可为空	典型用途
值传递	✓	✗	✗	小对象
指针传递	✗	✓	✓	C 风格接口
引用传递	✗	✓	✗	常规传参
const 引用	✗	✗	✗	大对象、接口
右值引用	✗	✓	✗	移动语义

一句话：

C++ 推荐 `const` 引用传参，大对象避免拷贝；右值引用用于移动语义。

typedef 与 using 的区别

基本用法

两者都用于 **类型别名**（type alias）。

```
typedef int MyInt1;  
using MyInt2 = int;
```

功能相同，但 **using** 语法更直观。

语法差异

typedef（旧语法）

类型写在后面，复杂类型可读性差：

```
typedef int* Ptr1;           // 指针  
typedef void (*Func)(int, int); // 函数指针
```

using（现代语法）

更符合“别名 = 类型”的直觉：

```
using Ptr2 = int*;  
using Func2 = void(*)(int, int);
```

模板别名（核心区别）

typedef 不能直接定义模板别名

必须写成复杂的嵌套形式：


```
template<typename T>
struct VecAlias {
    typedef std::vector<T> type;
};
VecAlias<int>::type v;  // 使用繁琐
```

using 可以直接定义模板别名（C++11）

语法简洁、可读性强：

```
template<typename T>
using Vec = std::vector<T>;

Vec<int> v;  // 直接使用
```

一句话：

using 支持模板别名，typedef 不支持，这是两者最大的区别。

可读性与可维护性

特性	typedef	using
语法直观性	较差	很好
复杂类型可读性	差	好
模板别名	✗ 不支持	✓ 强力支持
推荐程度	旧代码兼容	现代 C++ 推荐

使用建议

- 现代 C++（C++11+）推荐使用 using
- typedef 主要用于：
 - 兼容旧代码（C++98）
 - 某些 C 风格接口

一句话：

现代 C++ 中 using 完全优于 typedef，尤其在模板场景下。

野指针与悬挂指针

野指针（Wild Pointer）

定义

指向 **未知内存** 或 **未初始化** 的指针。

```
int *p;    // 未初始化 → 野指针
*p = 10;   // 未定义行为
```

产生原因

- 指针未初始化
- 指针被随意赋值
- 指针越界导致指向未知区域

危害

- 可能修改任意内存
- 程序随机崩溃

避免方法

```
int *p = NULL;
```

悬挂指针（Dangling Pointer）

定义

指向 **已被释放或已失效内存** 的指针。

```
int *p = malloc(sizeof(int));
free(p);
*p = 10; // 悬挂指针访问
```

产生原因

- free 后继续使用
- 返回局部变量地址
- 指向生命周期已结束的对象

危害

- 访问非法内存
- 难以复现的随机错误

避免方法

```
free(p);  
p = NULL;
```

核心区别（高频面试题）

指针类型	指向的内存	典型原因	是否已释放
野指针	未知/未初始化	未初始化、越界	✗ 未释放
悬挂指针	已释放/已失效	free 后继续用、返回局部变量地址	✓ 已释放

一句话：

野指针 = 未初始化；悬挂指针 = 用了已经释放的内存。

常见错误示例（悬挂指针）

```
int* func() {  
    int a = 10;  
    return &a; // ✗ 返回局部变量地址  
}
```

防范建议

- 指针定义即初始化
- free 后立即置 NULL
- 使用工具（ASan、Valgrind）
- C++ 中尽量使用智能指针（unique_ptr/shared_ptr）